

ga_golden_output_real_ids (golden-output, real repo data)

The task's cited "120 existing blocks" figure is STALE against this repo's real current state, verified directly (§11.4.273 — never trusted blindly): `grep -c "^CM-COVENANT-" constitution/scripts/gates/gate_ledger_baseline.txt` returns just the count **402** (the whole gate ledger, not the propagation family specifically); the REAL `CM-COVENANT-*-PROPAGATION` wrapper-script count on disk is **83** individual `.sh` files (verified: `ls constitution/scripts/gates/cm_covenant_114_*_propagation.sh | grep -vc mutation_test`), and the data-pack `covenant_propagation_anchors.tsv` that already drives the consolidated `covenant_propagation_suite.sh` batch runner has **157** rows. This fixture uses the REAL current figures, not the task text's "120".

`_shared/real_sample/` freezes 5 REAL propagation-gate files from the live repo (`cm_covenant_114_{162,167,176,187,190}_propagation.sh`), plus their real `lib/covenant_propagation_engine.sh` + `lib/pointer_carrier.sh` dependencies, all copied verbatim — never hand-edited) as a self-contained, genuinely-runnable golden-output sample.

Hand-verified during authoring (real execution against the live repo, `CONSUMER_ROOT=<repo-root>`, exit codes captured correctly — NOT via a pipeline that would silently discard them, the exact §11.4.201(7)(c) footgun this project's constitution documents):

id	clas- sifica- tion	real exit code (at authoring time)
162	ex- ecut- ing	1 (FAIL — anchor-block divergence)
167	ex- ecut- ing	1 (FAIL — 55 carriers missing §11.4.167; this file is a STAN-DALONE near-duplicate of 162's logic, NOT a thin-wrapper via the shared engine — still genuinely <code>executing</code> , just a different code shape, itself a candidate §11.4.251 finding <code>gate_audit</code> 's DUPLICATE-detection sub-feature should surface, though this fixture only asserts classification, not duplicate-pairing)
176	ex- ecut- ing	0 (PASS)
187	ex- ecut- ing	1 (FAIL — anchor-block divergence)
190	ex- ecut- ing	0 (PASS)

`gate_audit.py classify` MUST report ALL 5 as `executing` (none vacuous, none named-only — every one has a genuinely content-driven fail path, independently confirmed by the real exit codes above). This fixture asserts classification only; exact verdict-reproduction against a live, changing repo tree is out of scope (the frozen copies' `lib/` deps and target carriers are what they were at authoring time — a live re-run against the CURRENT tree may show different PASS/FAIL verdicts as the repo evolves, which is expected and not a fixture failure).